Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 "Компьютерные науки и прикладная математика"
Кафедра №806 "Вычислительная математика и программирование"

Лабораторная работа №2 по курсу
«Операционные системы»

Группа: М8О-208БВ-25
Студент: Андрушко К.М.
Преподаватель: Бахарев В.Д.
Оценка: _____
Дата: 26.10.2026

Москва, 2026

Постановка задачи

Цель работы: приобретение практических навыков в управлении потоками в ОС и в обеспечении синхронизации между потоками.

Задание: составить многопоточную программу на языке Си, используя стандартные средства ОС (POSIX Threads). Максимум одновременно работающих потоков задаётся ключом запуска. Нужно показать число потоков средствами ОС и исследовать, как ускорение и эффективность зависят от входных данных и числа потоков, а результаты объяснить.

Вариант 1: отсортировать массив целых чисел при помощи битонической сортировки.

Алгоритм

Битоническая последовательность - это последовательность, которая сначала возрастает, а потом убывает. Ключевое свойство: если элементы битонической последовательности длины n попарно сравнить на расстоянии n/2 и при необходимости обменять так, чтобы слева оказался меньший элемент, то получатся две половины, каждая из которых снова битоническая, причём все элементы левой половины не больше элементов правой. Повторяя эту операцию с расстояниями n/4, n/8, ..., 1, получаем отсортированную последовательность.

Последовательная версия (sort_seq) выполняет все шаги в одном потоке.

Параллельная версия (sort_par). Внутри одного шага все пары сравнений независимы, поэтому массив делится на p равных кусков, и каждый кусок обрабатывает свой поток (p - 1 созданный поток и главный). После шага главный поток ждёт остальных через pthread_join: следующий шаг нельзя начинать, пока не закончен предыдущий.

Ограничение числа потоков. Ключ -t задаёт максимум потоков p, главный поток тоже считается. Текущее число потоков (running) и максимум

(peak) общие, поэтому они защищены мьютексом, иначе будет гонка данных. Значение peak выводится в конце.

Что сортируется. Сортируется массив из n случайных чисел типа int, созданных функцией rand().

Проверка корректности. Результаты последовательной и параллельной версий сравниваются между собой, а также проверяется, что массив неубывающий.

Демонстрационный режим (-d): на первом шаге каждый поток засыпает на 5 секунд, чтобы все потоки были живы одновременно и их можно было посчитать средствами ОС. Время в этом режиме не используется.

Метрики: ускорение и эффективность

Ускорение:

$$S(p) = Ts / Tp(p),$$

где Ts - время последовательной версии, Tp(p) - время параллельной версии на p потоках.

Эффективность:

$$E(p) = S(p) / p = Ts / (p \cdot Tp(p)),$$

где p - число потоков.

Время замеряется с помощью clock_gettime (CLOCK_MONOTONIC) в миллисекундах.

Код программы

lab2.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <limits.h>
#include <unistd.h>
#include <time.h>
#include <pthread.h>


static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
static int running = 1;
static int peak = 1;
static int create_errors = 0;
```

```
static void enter thread(void) {
    pthread mutex lock(&m);
    running++;
    if (running > peak) peak = running;
    pthread_mutex_unlock(&m);
}


static void leave_thread(void) {
    pthread mutex lock(&m);
    running--;
    pthread_mutex_unlock(&m);
}


static double now ms(void) {
    struct timespec ts;
    clock gettime(CLOCK MONOTONIC, &ts);
    return ts.tv_sec * 1000.0 + ts.tv_nsec / 1e6;
}


static void compare_range(int *a, size_t k, size_t j, size_t from, size_t to) {
    for (size_t i = from; i < to; i++) {
        if (i % (2 * j) < j) {
            size_t l = i + j;
            int asc = ((i / k) % 2 == 0);
            int need_swap;
            if (asc) need swap = (a[i] > a[l]);
            else     need swap = (a[i] < a[l]);
            if (need_swap) {
                int t = a[i];
                a[i] = a[l];
                a[l] = t;
            }
        }
    }
}


static void sort seq(int *a, size t N) {
    for (size_t k = 2; k <= N; k = k * 2)
        for (size t j = k / 2; j > 0; j = j / 2)
            compare_range(a, k, j, 0, N);
}


typedef struct {
    int *a;
    size t k, j, from, to;
    bool demo;
} task_t;


static void *thread func(void *arg) {
    task_t *t = arg;
    enter_thread();
    if (t->demo) sleep(5);
    compare_range(t->a, t->k, t->j, t->from, t->to);
    leave thread();
    return NULL;
}


static void sort par(int *a, size t N, int p, bool demo) {
    pthread_t *threads = malloc(p * sizeof(pthread_t));
    task t *tasks = malloc(p * sizeof(task t));
    bool *created = malloc(p * sizeof(bool));


    for (size_t k = 2; k <= N; k = k * 2) {
        for (size t j = k / 2; j > 0; j = j / 2) {
            bool first_step = demo && k == 2;
            for (int t = 0; t < p; t++) {
                tasks[t].a = a;
                tasks[t].k = k;
                tasks[t].j = j;
                tasks[t].from = N * t / p;
                tasks[t].to = N * (t + 1) / p;
                tasks[t].demo = first_step;
```

```
            }
            for (int t = 0; t < p - 1; t++) {
                created[t] = (pthread_create(&threads[t], NULL, thread_func, &tasks[t]) == 0);
                if (!created[t]) {
                    create_errors++;
                    compare_range(a, k, j, tasks[t].from, tasks[t].to);
                }
            }
            if (first_step) sleep(5);
            compare_range(a, k, j, tasks[p - 1].from, tasks[p - 1].to);
            for (int t = 0; t < p - 1; t++)
                if (created[t]) pthread_join(threads[t], NULL);
        }
    }
    free(threads);
    free(tasks);
    free(created);
}


static int is_sorted(const int *a, size_t n) {
    for (size_t i = 1; i < n; i++)
        if (a[i - 1] > a[i]) return 0;
    return 1;
}


static void print_arr(const char *title, const int *a, size_t n) {
    size_t k = n < 10 ? n : 10;
    printf("%s\n начало:", title);
    for (size_t i = 0; i < k; i++) printf(" %d", a[i]);
    printf("\n конец: ");
    for (size_t i = n - k; i < n; i++) printf(" %d", a[i]);
    printf("\n");
}


int main(int argc, char **argv) {
    long n = 10000000;
    int p = 1;
    bool demo = false;
    bool show = false;
    int o;
    while ((o = getopt(argc, argv, "n:t:dp")) != -1) {
        if (o == 'n') n = atol(optarg);
        else if (o == 't') p = atoi(optarg);
        else if (o == 'd') demo = true;
        else if (o == 'p') show = true;
        else return 1;
    }
    if (p < 1) p = 1;
    if (n < 2) {
        fprintf(stderr, "Размер массива должен быть не меньше 2\n");
        return 1;
    }


    size_t N = 1;
    while (N < (size_t)n) N = N * 2;


    int *orig = malloc(N * sizeof(int));
    int *a = malloc(N * sizeof(int));
    int *b = malloc(N * sizeof(int));
    if (!orig || !a || !b) {
        fprintf(stderr, "Не хватает памяти\n");
        return 1;
    }
    srand(1);
    for (long i = 0; i < n; i++) orig[i] = rand();
    for (size_t i = n; i < N; i++) orig[i] = INT_MAX;


    printf("PID=%d, логических ядер=%ld, n=%ld, потоков (лимит)=%d\n",
           getpid(), sysconf(_SC_NPROCESSORS_ONLN), n, p);
    if (show) print_arr("Массив до сортировки:", orig, n);


    memcpy(a, orig, N * sizeof(int));
    double t0 = now_ms();
    sort_seq(a, N);
```

```
    double ts = now_ms() - t0;


    memcpy(b, orig, N * sizeof(int));
    t0 = now_ms();
    sort_par(b, N, p, demo);
    double tp = now_ms() - t0;


    if (show) print_arr("Массив после параллельной сортировки:", b, n);


    bool ok = memcmp(a, b, n * sizeof(int)) == 0 && is_sorted(b, n);
    double sp = ts / tp;
    double ef = sp / p;


    printf("Проверка: %s, пик потоков: %d\n", ok ? "OK" : "ОШИБКА", peak);
    if (create_errors > 0)
        printf("Ошибок создания потоков: %d (их работу выполнил главный поток)\n",
create_errors);
    printf("Время последовательной версии: %.2f мс\n", ts);
    printf("Время параллельной версии: %.2f мс\n", tp);
    printf("Ускорение: %.3f\n", sp);
    printf("Эффективность: %.3f\n", ef);


    free(orig);
    free(a);
    free(b);
    return ok ? 0 : 2;
}
```

Протокол работы программы

Среда: процессор Apple M1, число физических ядер 8, число логических ядер 8, ОС macOS 27.0.1.

Тест 1. Корректность

Команда: ./lab2 -n 10000000 -t 4

```
PID=2462, логических ядер=8, n=10000000, потоков (лимит)=4
Проверка: OK, пик потоков: 4
Время последовательной версии: 4013.32 мс
Время параллельной версии: 1736.53 мс
Ускорение: 2.311
Эффективность: 0.578
```

Проверка «OK» значит, что результаты обеих версий совпали и массив отсортирован. Пик потоков равен лимиту.

Тест 2. Число потоков средствами ОС

Программа запущена в демонстрационном режиме (-d): на первом шаге каждый поток 5 секунд ждёт, чтобы все потоки были живы одновременно. В это время средствами ОС получено число потоков процесса.

Команда: ./lab2 -n 2000000 -t 8 -d & sleep 2; ps -M $!; wait

```
[1] 2464
PID=2464, логических ядер=8, n=2000000, потоков (лимит)=8
USER    PID   TT   %CPU STAT PRI     STIME     UTIME COMMAND
flaym  2464 s000    0,3 S    31T   0:00.01   0:00.51 ./lab2 -n 2000000 -t 8 -d
       2464         0,0 S    31T   0:00.00   0:00.00
       2464         0,0 S    31T   0:00.00   0:00.00
       2464         0,0 S    31T   0:00.00   0:00.00
       2464         0,0 S    31T   0:00.00   0:00.00
       2464         0,0 S    31T   0:00.00   0:00.00
       2464         0,0 S    31T   0:00.00   0:00.00
Проверка: OK, пик потоков: 8
Время последовательной версии: 479.67 мс
Время параллельной версии: 5212.40 мс
Ускорение: 0.092
Эффективность: 0.012
[1]  + done       ./lab2 -n 2000000 -t 8 -d
```

Команда ps -M выводит каждый поток отдельной строкой. Их 8, как и задано ключом -t 8 (7 созданных потоков и главный), и это совпадает с «пиком потоков» из вывода программы. В первой строке указана команда (главный поток), у остальных семи колонка с командой пустая.

Тест 3. Что именно сортируется

Ключ -p выводит по 10 первых и последних элементов массива до и после параллельной сортировки (вне замеров времени).

Команда: ./lab2 -n 10000000 -t 4 -p

```
PID=2467, логических ядер=8, n=10000000, потоков (лимит)=4
Массив до сортировки:
 начало: 16807 282475249 1622650073 984943658 1144108930 470211272 101027544 1457850878
1458777923 2007237709
 конец:  1162828487 1557193309 364738374 1239523280 2076391060 1295281670 757298051 1920251035
1274898129 1768507984
Массив после параллельной сортировки:
 начало: 275 335 857 884 897 1003 1042 1073 1203 1282
 конец:  2147481981 2147482033 2147482303 2147482433 2147482521 2147482693 2147482925 2147483235
2147483321 2147483531
Проверка: OK, пик потоков: 4
Время последовательной версии: 4000.32 мс
Время параллельной версии: 1684.55 мс
Ускорение: 2.375
Эффективность: 0.594
```

Исследование ускорения и эффективности

Все измерения проведены на массиве из n = 10 000 000 случайных чисел типа int, каждое значение - среднее по 5 запускам. Процессор Apple M1, логических ядер 8. Ts ≈ 4000 мс.

Зависимость от числа потоков

Число потоков	Время исполнения (мс)	Ускорение	Эффективность
1	4008,62	0,995	0,995
2	3157,55	1,259	0,630
4	1671,28	2,380	0,595
6	1813,85	2,199	0,366
8 (= числу ядер)	1513,75	2,667	0,333
16	1445,97	2,804	0,175
128	1500,22	2,703	0,021
1024	3756,38	1,079	0,001

Таблица 1. Время, ускорение и эффективность для n = 10 000 000. Потоков меньше числа ядер: 1, 2, 4, 6; равно числу ядер: 8; сильно больше: 16, 128, 1024.

Зависимость от размера входных данных

Размер массива n	Ts (мс)	Tp (мс)	Ускорение	Эффективность
100 000	23,92	18,26	1,310	0,164
1 000 000	216,62	86,49	2,505	0,313
10 000 000	4007,93	1573,06	2,548	0,318

Таблица 2. Зависимость от размера массива при p = 8 (равно числу логических ядер).

Вывод

В работе реализованы последовательная и параллельная битоническая сортировка на POSIX Threads. Число потоков ограничивается ключом -t и проверяется через ps. Синхронизация нужна в двух местах: pthread_join не даёт начать следующий шаг раньше времени, а мьютекс защищает общие счётчики потоков.

Результаты. При p = 1 ускорение 0,995 (потоки не создаются). С ростом p до числа ядер оно растёт: 1,259 при p = 2, 2,380 при p = 4 и 2,667 при p = 8, но до идеального S = p не доходит. Причины: потоки создаются заново на каждом из 300 шагов, сортировка упирается в память, а у Apple M1 ядра разные (4 производительных и 4 энергоэффективных). Видимо, поэтому при p = 6 ускорение (2,199) даже ниже, чем при p = 4: шаг заканчивается, когда закончит самый медленный поток.

При p = 16 и 128 ускорение почти не меняется (2,804 и 2,703): ядер больше нет. При p = 1024 оно падает до 1,079, потому что на создание и переключение потоков уходит много времени. Эффективность E = S / p падает с 0,995 при p = 1 до 0,001 при p = 1024.

От размера массива. Ускорение растёт с размером: 1,310 при n = 10^5, 2,505 при n = 10^6 и 2,548 при n = 10^7. На маленьком массиве создание потоков занимает почти столько же времени, сколько сама сортировка. Для n = 10^7 в таблице 1 вышло 2,667: разница из-за разброса измерений.

Лучше брать число потоков около числа ядер: меньше - процессор простаивает, больше - программа только замедляется.